Software Re-Engineering

Lecture 14

Reverse engineering II – Introduction to ARM, moving btw Assembly and c code

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Advanced RISC Machine

- Arm is RISC (Reduced Instruction Set Computing) based, while x86 is CISC (Complex Instruction Set Computing).
- Arm’s CPU instructions are reasonably atomic, with a very close correlation between the number of instructions and micro-ops. CISC, by comparison, offers many more instructions, many of which execute multiple operations (such as optimized math and data movement).
- This leads to better performance but more power consumption in decoding these complex instructions.

LDR and STR

```
01: 03 68          LDR          R3, [R0] ; R3 = *R0
02: 23 60          STR          R3, [R4] ; *R4 = R3;
```

For the instruction in line 1, R0 is the base register and R3 is the destination; it loads the word value at address R0 into R3.

In line 2, R4 is the base register and R3 is the destination; it takes the value in R3 and stores at the memory address R4.

At a fundamental level, the LDR/STR instructions take a base register and an offset

STR Ra, [Rb, imm]
Rb is the base register, and imm is the offset to be added to Rb

functional assembly program:

```
.data          /* the .data section is dynamically created and its addresses cannot be easily predicted */
var1: .word 3  /* variable 1 in memory */
var2: .word 4  /* variable 2 in memory */

.text          /* start of the text (code) section */
.global _start

_start:
    ldr r0, adr_var1  @ load the memory address of var1 via label adr_var1 into R0
    ldr r1, adr_var2  @ load the memory address of var2 via label adr_var2 into R1
    ldr r2, [r0]      @ load the value (0x03) at memory address found in R0 to register R2
    str r2, [r1]      @ store the value found in R2 (0x03) to the memory address found in R1
    bkpt

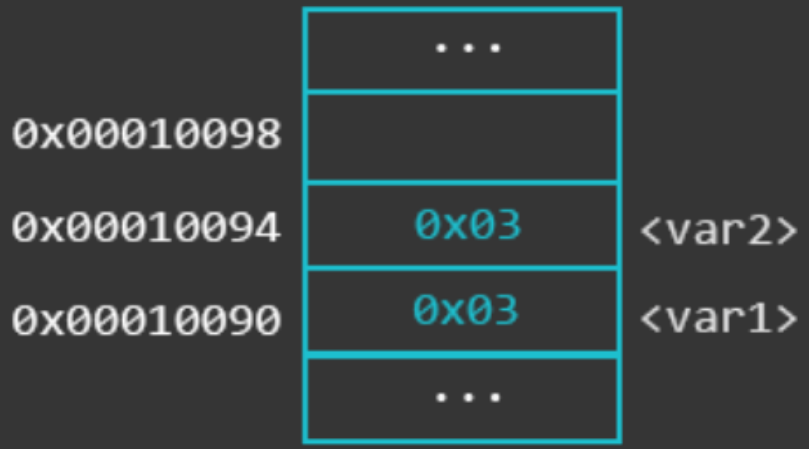
adr_var1: .word var1  /* address to var1 stored here */
adr_var2: .word var2  /* address to var2 stored here */
```

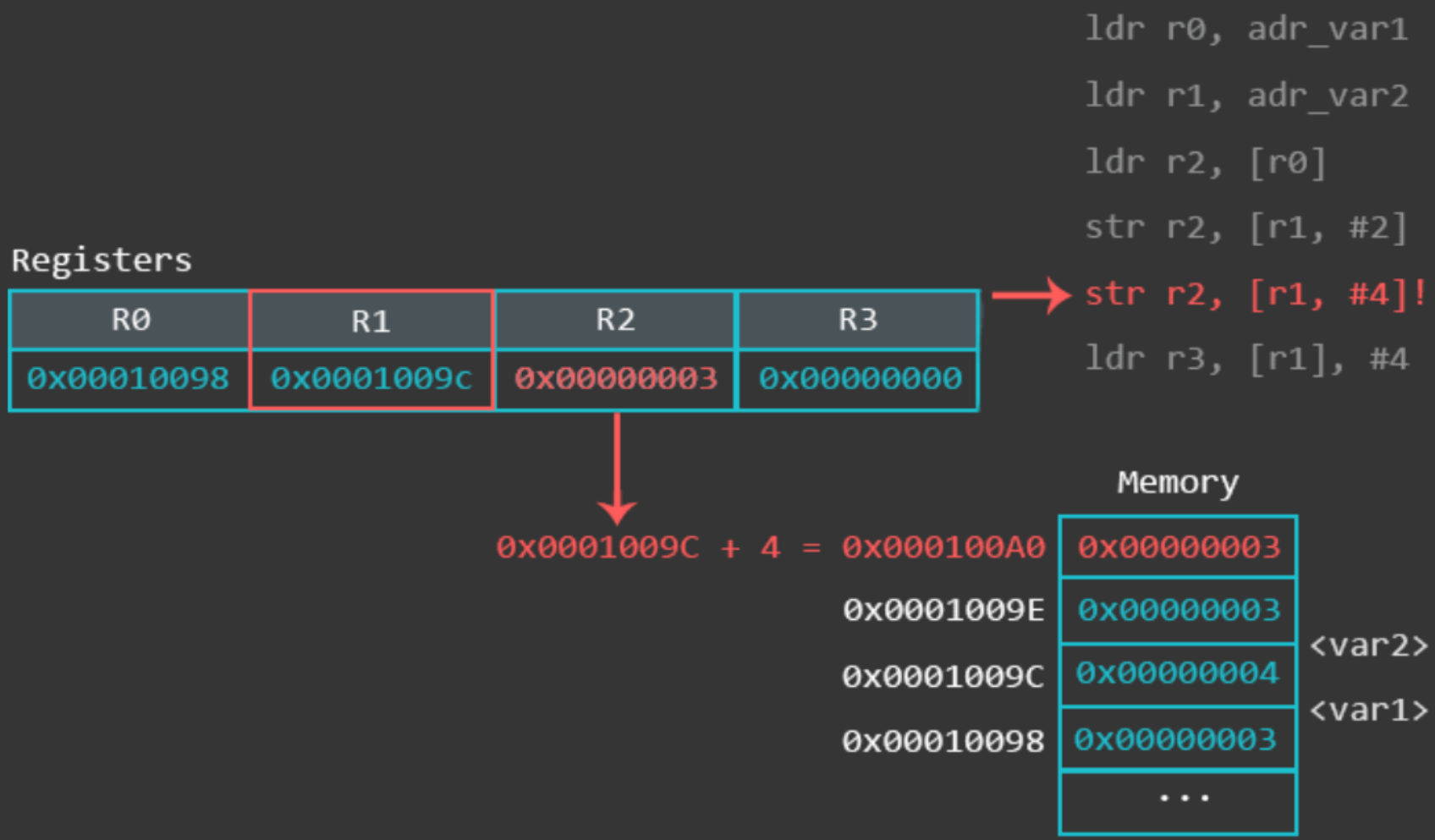
```
ldr   r0, addr_var1
ldr   r1, addr_var2
ldr   r2, [r0]
str   r2, [r1]
```

Registers

R0	R1	R2
0x00010090	0x00010094	0x00000003

Memory

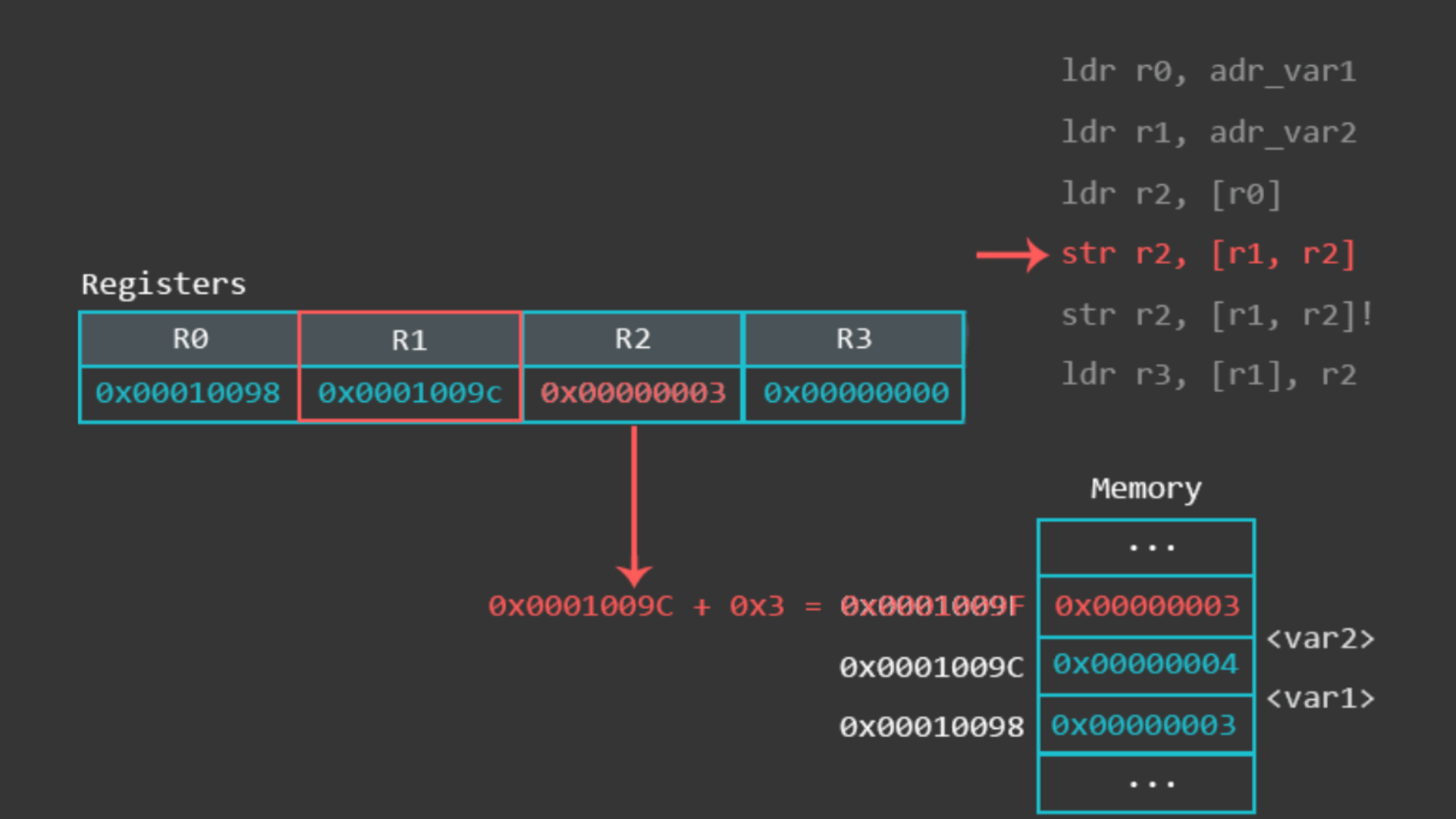



ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, #2]
str r2, [r1, #4]!
ldr r3, [r1], #4
Registers
R0
R1
R2
R3
0x00010098
0x0001009c
0x00000003
0x00000000
0x0001009C + 4 = 0x000100A0
Memory
0x00000003
0x0001009E
0x00000003
<var2>
0x0001009C
0x00000004
<var1>
0x00010098
0x00000003
...

Register as the offset.

```
STR    Ra, [Rb, Rc]
LDR    Ra, [Rb, Rc]
```

```
_start:
    ldr r0, adr_var1  @ load the memory address of var1 via label adr_var1 to R0
    ldr r1, adr_var2  @ load the memory address of var2 via label adr_var2 to R1
    ldr r2, [r0]      @ load the value (0x03) at memory address found in R0 to R2
    str r2, [r1, r2]  @ address mode: offset. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register unmodified.
    str r2, [r1, r2]! @ address mode: pre-indexed. Store value found in R2 (0x03) to the memory address found in R1 with the offset R2 (0x03). Base register modified: R1 = R1+R2.
    ldr r3, [r1], r2  @ address mode: post-indexed. Load value at memory address found in R1 to register R3. Then modify base register: R1 = R1+R2.
    bx lr
```


ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, r2]
str r2, [r1, r2]!
ldr r3, [r1], r2
Registers
R0
R1
R2
R3
0x00010098
0x0001009c
0x00000003
0x00000000
0x0001009C + 0x3 = 0x0001009F
Memory
...
0x00000003
<var2>
0x0001009C
0x00000004
<var1>
0x00010098
0x00000003
...

Scaled register as the offset

LDR Ra, [Rb, Rc, <shifter>]
STR Ra, [Rb, Rc, <shifter>]

```
_start:
    ldr r0, adr_var1        @ load the memory address of var1 via label adr_var1 to R0
    ldr r1, adr_var2        @ load the memory address of var2 via label adr_var2 to R1
    ldr r2, [r0]          @ load the value (0x03) at memory address found in R0 to R2
    str r2, [r1, r2, LSL#2]  @ address mode: offset. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register (R1) unmodified.
    str r2, [r1, r2, LSL#2]! @ address mode: pre-indexed. Store the value found in R2 (0x03) to the memory address found in R1 with the offset R2 left-shifted by 2. Base register modified: R1 = R1 + R2<<2
    ldr r3, [r1], r2, LSL#2  @ address mode: post-indexed. Load value at memory address found in R1 to the register R3. Then modifiy base register: R1 = R1 + R2<<2
```

```
ldr r0, adr_var1
ldr r1, adr_var2
ldr r2, [r0]
str r2, [r1, r2, LSL#2]
[...]
```

Registers

R0	R1	R2
0x00010098	0x0001009c	0x00000003

1. 0x00000003 (hex) = 0000 0011 (binary)
2. 0000 0011 LSL#2 = 0000 1100
3. 0000 1100 = 0xC (hex)
4. 0x0001009c + 0xC = 0x00100a8

   base offset

Memory

0x000100a8	0x00000003	
	...	
0x0001009c	0x00000004	<var2>
0x00010098	0x00000003	<var1>
	...	

SUMMARY

Three offset modes in LDR/STR:

- offset mode uses an immediate as offset

```
ldr   r3, [r1, #4]
```

- offset mode uses a register as offset

```
ldr   r3, [r1, r2]
```

- offset mode uses a scaled register as offset

```
ldr   r3, [r1, r2, LSL#2]
```

Functions and Function Invocation

The Basics of Function Calls

At its core, calling a function in assembly involves several key steps:

1. **Preparing Arguments**: Load function arguments into specified registers or onto the stack.
2. **Branching**: Use a branch instruction to jump to the function's entry point.
3. **Function Execution**: The called function executes, possibly modifying registers or the stack.
4. **Returning**: The function returns control to the caller, potentially passing back a return value.
5. **Clean-Up**: Any necessary clean-up tasks are performed, such as restoring registers or adjusting the stack.

Function arguments can be passed in registers or on the stack, depending on the calling convention. Commonly used registers for argument passing in ARM are R0 to R3. If more arguments are required, additional arguments are typically passed on the stack.

Here’s an example of passing arguments in registers:

```
MOV     R0, #5        ; Argument 1: R0 = 5
    MOV     R1, #10       ; Argument 2: R1 = 10
    BL      my_function   ; Call the function
```

Branching

The branch instruction BL (Branch with Link) is used to jump to the function's entry point. It stores the return address in the link register (LR or R14) so that the function can return control to the caller.

```
BL      my_function    ; Call the function, LR stores the return address
```

Function Execution

Within the called function, arguments are accessed based on the calling convention. The function uses these arguments as needed, potentially modifying registers, and executes its code.

```
my_function:
    ADD     R2, R0, R1   ; Add arguments R0 and R1, result in R2
    ...
    BX      LR           ; Return using the value in LR
```

Returning

The BX (Branch and Exchange) instruction is used to return control to the caller. It branches to the address stored in the link register (LR), which holds the return address.

```
my_function:
    ...
    BX      LR          ; Return using the value in LR
```

Clean-Up

After returning from the function, any necessary clean-up tasks must be performed. This may include restoring registers or adjusting the stack, depending on the calling convention and the function’s requirements.

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

Entering two arguments in R0 and R1 respectively. Registers R0-R3 are for arguments. If more than 4 arguments then the arguments are pushed to stack

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

Push to save state. Otherwise when add function updates r0 and r1 and the control gets back to the instruction /line of code present after func call; we missed out to original values of r0 and r1. the values have been overwritten

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

BL: branch with link is used to call a routine

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

This adds up r0 and r1 and puts the result in r0. The result is always stored in r0.

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

BX branch and exchange causes a branch to an address specified by register. Link register LR stores address of the instruction mentioned after function call…i.e. address of instruction “mov r2,r0”

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

This is done to safe state. i.e. save value r0 returned by function. Because in upcoming lines we are going to reload original values of r0 and r1

```
int add(int a, int b)
{
        return a+b;
}
int main(void)
{
        add(1,2);
        return 0;
}
```

```
_start:
        mov r0, #1
        mov r1, #2

        push {r0,r1}
        bl add
        mov r2, r0
        pop {r0, r1}

Add:
        add r0, r0, r1
        bx lr
```

reload original values of r0 and r1… retrieved the state

```
int main()
{
 int res = 0;
 int a = 1;
 int b = 2;
 res = max(a, b);
 return res;
}
```

```
main:
        push   {r11, lr}    /* Start of the prologue. Saving Frame Pointer and LR onto the stack
*/
        add    r11, sp, #0  /* Setting up the bottom of the stack frame */
        sub    sp, sp, #16  /* End of the prologue. Allocating some buffer on the stack */
        mov    r0, #1       /* setting up local variables (a=1). This also serves as setting up the
first parameter for the max function */
        mov    r1, #2       /* setting up local variables (b=2). This also serves as setting up the
second parameter for the max function */
        bl     max          /* Calling/branching to function max */
        sub    sp, r11, #0  /* Start of the epilogue. Readjusting the Stack Pointer */
        pop    {r11, pc}    /* End of the epilogue. Restoring Frame pointer from the stack,
jumping to previously saved LR via direct load into PC */
```

```
int max(int a,int b)
{
 if(a<b)
 {
 return b;
 }
 else
 {
 return a;
 }
}
```

```
max:
push   {r11}        /* Start of the prologue. Saving Frame Pointer onto the stack */
add    r11, sp, #0  /* Setting up the bottom of the stack frame */
sub    sp, sp, #12  /* End of the prologue. Allocating some buffer on the stack */
cmp    r0, r1       /* Implementation of if(a<b) */
movlt  r0, r1       /* if r0 was lower than r1, store r1 into r0 */
add    sp, r11, #0  /* Start of the epilogue. Readjusting the Stack Pointer */
pop    {r11}        /* restoring frame pointer */
bx     lr           /* End of the epilogue. Jumping back to main via LR register */
```

```
int main() {
   int max = 0;
   int a = 2;
   int b = 3;
   if(a < b) {
    max = b;
   }
   else {
    max = a;
   }
   return max;
}
```

```
.global main

main:
        mov     r1, #2     /* setting up initial variable a */
        mov     r2, #3     /* setting up initial variable b */
        cmp     r1, r2     /* comparing variables to determine
which is bigger */
        blt     r1_lower   /* jump to r1_lower in case r2 is >*/
        mov     r0, r1     /* if branching/jumping did not occur, r1
is bigger (or the same) so store r1 into r0 */
        b       end        /* proceed to the end */
r1_lower:
        mov r0, r2         /* We ended up here because r1 was
smaller than r2, so move r2 into r0 */
        b end              /* proceed to the end */
end:
        bx lr              /* THE END */
```

```
int main() {
   int a = 0;
   while(a < 4) {
   a= a+1;
   }
   return a;
}
```

```
.global main

main:
        mov     r0, #0     /* setting up initial variable a */
loop:
        cmp     r0, #4     /* checking if a==4 */
        beq     end        /* proceeding to the end if a==4 */
        add     r0, r0, #1 /* increasing a by 1 if the jump to the
end did not occur */
        b loop             /* repeating the loop */
end:
        bx lr              /* THE END */
```